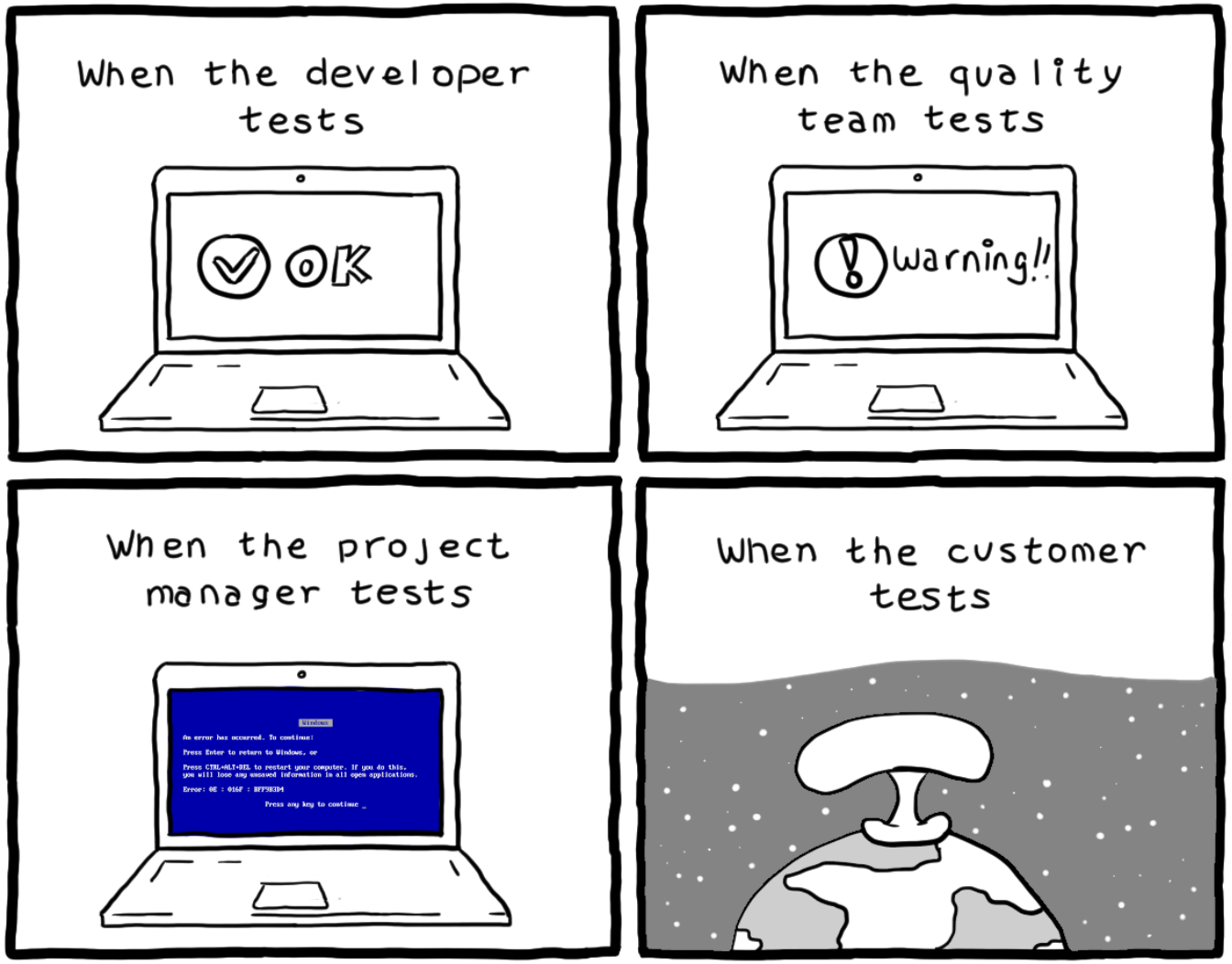
Langage C# - Tests unitaires

Animé par Sylvain Labasse

INTRODUCTION


When the developer tests
OK
When the quality team tests
warning!!
When the project manager tests
Windows
An error has occurred. To continue:
Press Enter to return to Windows, or
Press CTRL+ALT+DEL to restart your computer. If you do this, you will lose any unsaved information in all open applications.
Error: 0E : 016F : BFF9B3D4
Press any key to continue _
When the customer tests

EN FIN DE MODULE, VOUS SAUREZ…

Enumérer les critères de bons tests unitaires

Valider le bon fonctionnement d'une méthode/classe

Vérifier qu'une méthode/classe gère bien les cas d'erreurs

Faire émerger une conception grâce à TDD

Eventuellement, simuler les dépendances d'une classe

Pre-Requis

Public

SN2 - Développeur d’applications

Nécessaire

Fondamentaux du langage C#

Notion de classe

DEROULEMENT

Support

Copie des slides / Vidéos

Notes de cours

Ateliers

Evaluation

Ateliers sur 4

QCM (individuel)

0	1	2	3	4
Non remis	Hors sujet	<	=	>
		attentes		

L'ENVIRONNEMENT

Matériel

PC sous Windows/Mac + Connexion Internet

Logiciel

Visual Studio 2026

Langage C# - Tests unitaires

Tests unitaires

- Présentation
- Exigences
- Frameworks

MS Test

- Projet de test
- Organisation
- Exceptions

Langage C# - Tests unitaires

TDD

- Principe
- Red Green Refactor
- Emergence

Dépendances

- Problématique
- Prérequis
- Stubs vs Mocks

TESTS UNITAIRES

Objectifs

Utilité des tests unitaires

Contraintes de réalisation

Outils pour la mise en œuvre

TESTS UNITAIRES

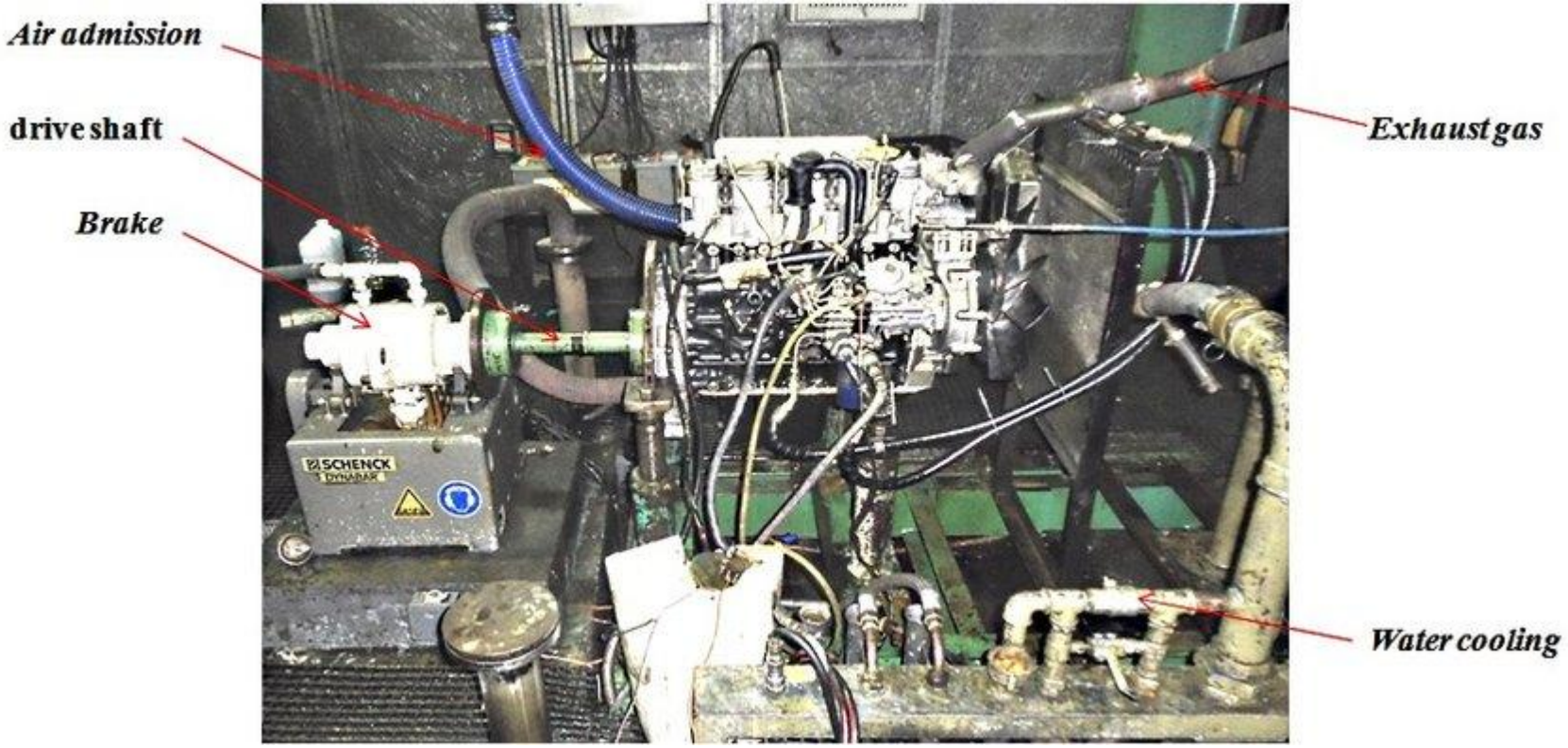
➔ Présentation

Exigences

Frameworks

Synthèse

PRESENTATION



Presentation

But

Validation de la classe

Quel que soit l’utilisation

Test

Boite noire

TESTS UNITAIRES

✓ Présentation

➔ Exigences

Frameworks

Synthèse

Exigences

Organisation

Projet de test indépendant du projet testé, mais même dépôt

1 classe testée = 1 classe de test

Critères

Autonome et déterministe

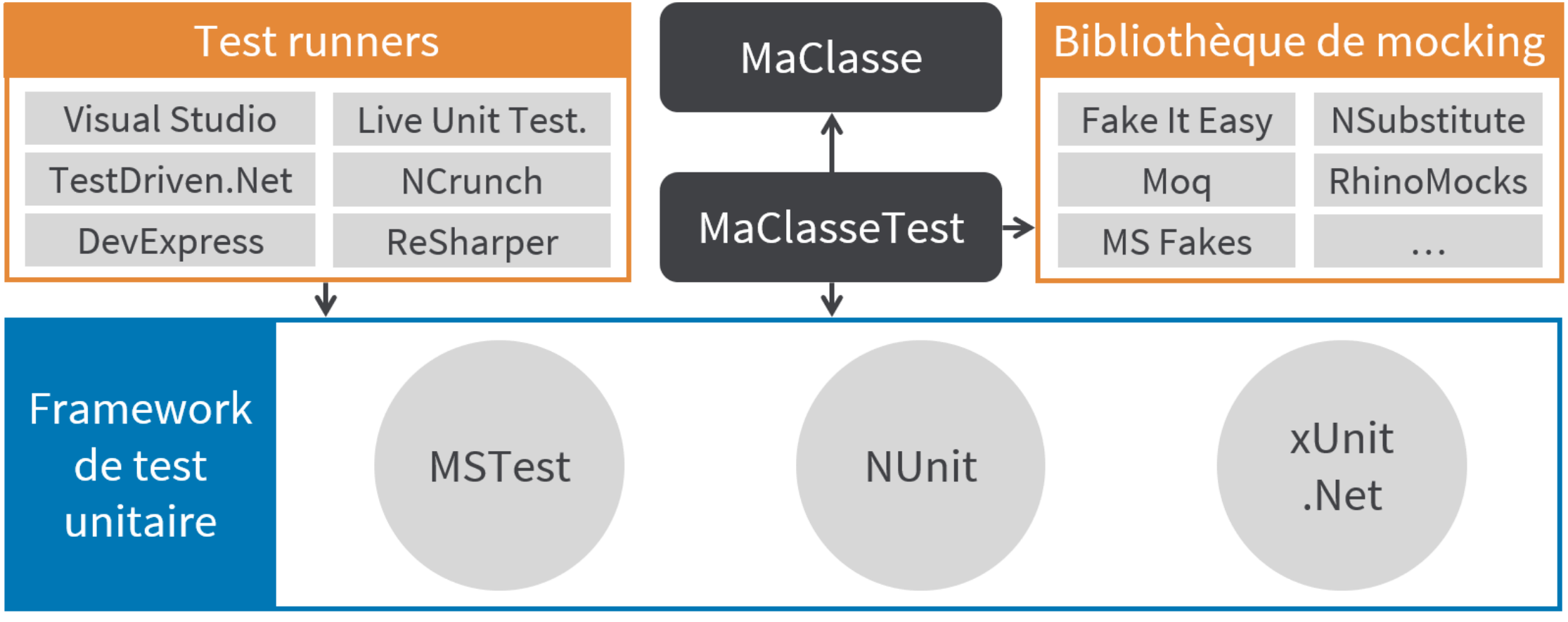
Facile à écrire et facile à lire

Exécution rapide

TESTS UNITAIRES

- ✓ Présentation
- ✓ Exigences
- ➔ Frameworks
- Synthèse

Frameworks


Test runners
Visual Studio
Live Unit Test.
TestDriven.Net
NCrunch
DevExpress
ReSharper
MaClasse
MaClasseTest
Bibliothèque de mocking
Fake It Easy
NSubstitute
Moq
RhinoMocks
MS Fakes
...
Framework de test unitaire
MSTest
NUnit
xUnit .Net

TESTS UNITAIRES

- ✓ Présentation
- ✓ Exigences
- ✓ Frameworks
- ➔ Synthèse

RESUME

Utilité des tests unitaires

Contraintes de réalisation

Outils pour la mise en œuvre

MS TEST

Objectifs

Premier projet de test

Organisation et conception des tests

Prise en main du framework

Gestion des erreurs

MS TEST

- ➔ Projet de test
- Organisation
- Exceptions
- Synthèse

Projet de test

Depuis Visual Studio

- Un projet de test indépendant
- Dans la même solution
- Choix du framework avec template

Référence dans le test

- Ajouter la référence (du projet testé dans le projet de test)
- Ajouter le/les using

MS TEST

- ✓ Projet de test
- ➔ Organisation
- Exceptions
- Synthèse

Cas de test

Unité de travail

Groupe de méthodes concourant à une action commune

Cas de test

Par unité de travail : Cas nominal, cas extrême, cas d'erreur

1 cas = 1 méthode de test

AAA

Arrange

Préparation du test

Notamment instance à tester

Act

Action à vérifier

Assert

Vérifier si résultat attendu = résultat constaté

Atelier 1 – Test simple

Sujet

Tester la classe Produit en vérifiant l'initialisation des prix d'achats et ventes et la bonne gestion des stocks avec rupture de stock. Attention, les membres PrixVente, EstEnRupture et Sortir peuvent être erronés ou incomplets : Profitez des tests pour compléter, corriger et vérifier ces méthodes.

Livrable

.cs du test unitaire.

MS TEST

- ✓ Projet de test
- ✓ Organisation
- ➔ Exceptions
- Synthèse

Exceptions

Cas d’erreur

Consiste à vérifier qu’une exception est bien levée.

Assertion

```
Action act = () => test.MethodeLevantLException();
Assert.Throws<TypeException>(act);
```

Vérification du message (rare)

```
var e = Assert.Throws<TypeException>(act);
Assert.AreEqual("…", e.Message); // rare
```

Atelier 2 – Module Shopping Cart

Sujet

Tester la classe Cart et, optionnellement, la classe Article, qui représentent respectivement un panier de site marchand et une entrée de ce panier. Appuyez-vous sur la documentation intégrée pour n'oublier aucun cas usuel, extrême et erreur. Factoriser les tests, si nécessaire.

Livrable

.cs du test (ou archive des deux .cs si Article est testé)

Resume

Premier projet de test

Organisation et conception des tests

Prise en main du framework

Gestion des erreurs

Validation des interactions

TDD

Objectifs

Avantages de la méthode

Etapes du processus de développement par les tests

Elaboration progressive d’une solution

TDD

- ➔ Principes
- Red Green Refactor
- Emergence
- Synthèse

Principes

Origines

Test Driven Development by example - 2003

Kent Beck, signataire manifeste agile

Avantages

Approche pragmatique et incrémentale

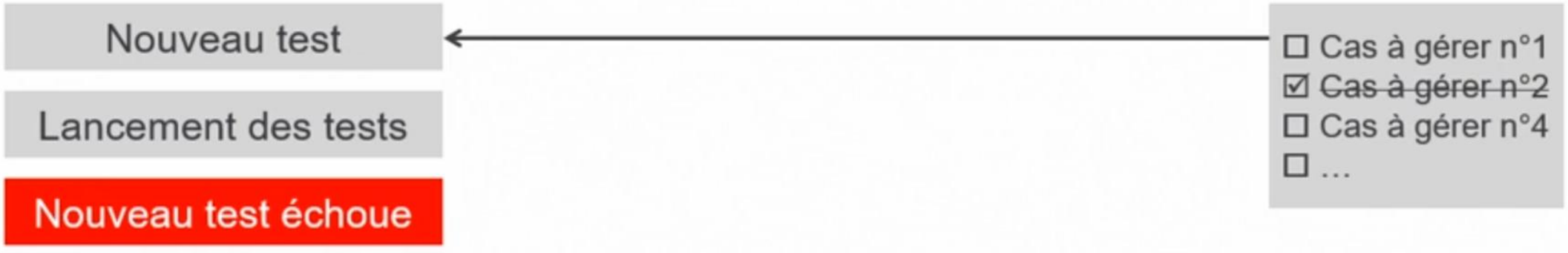
Test unitaire et non régression en premier

Spécifications et documentation à jour

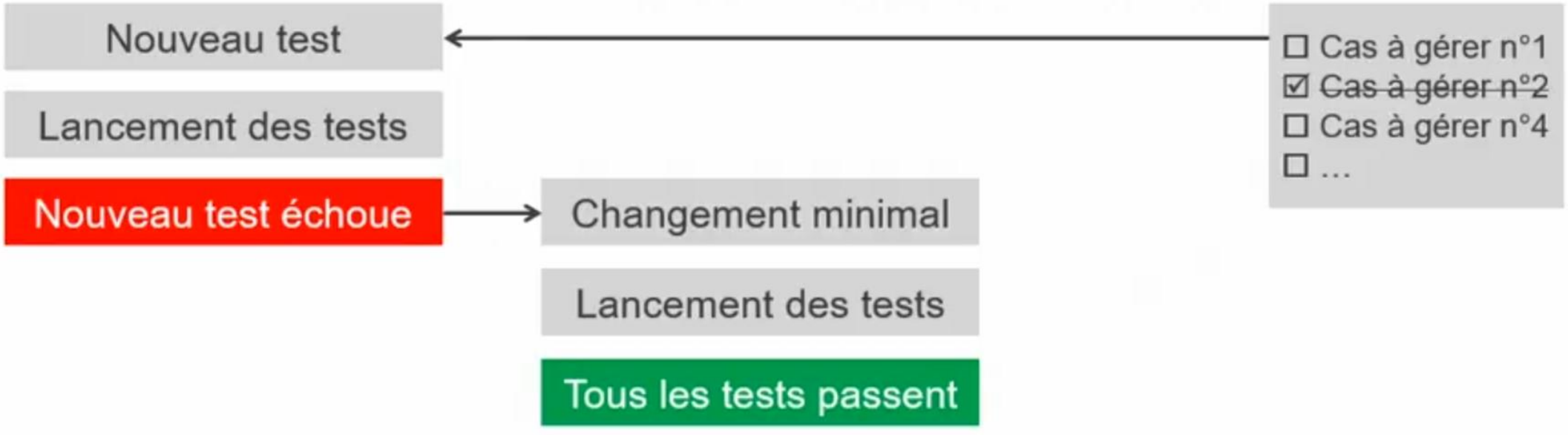
TDD

- ✓ Principes
- ➔ Red Green Refactor
- Emergence
- Synthèse

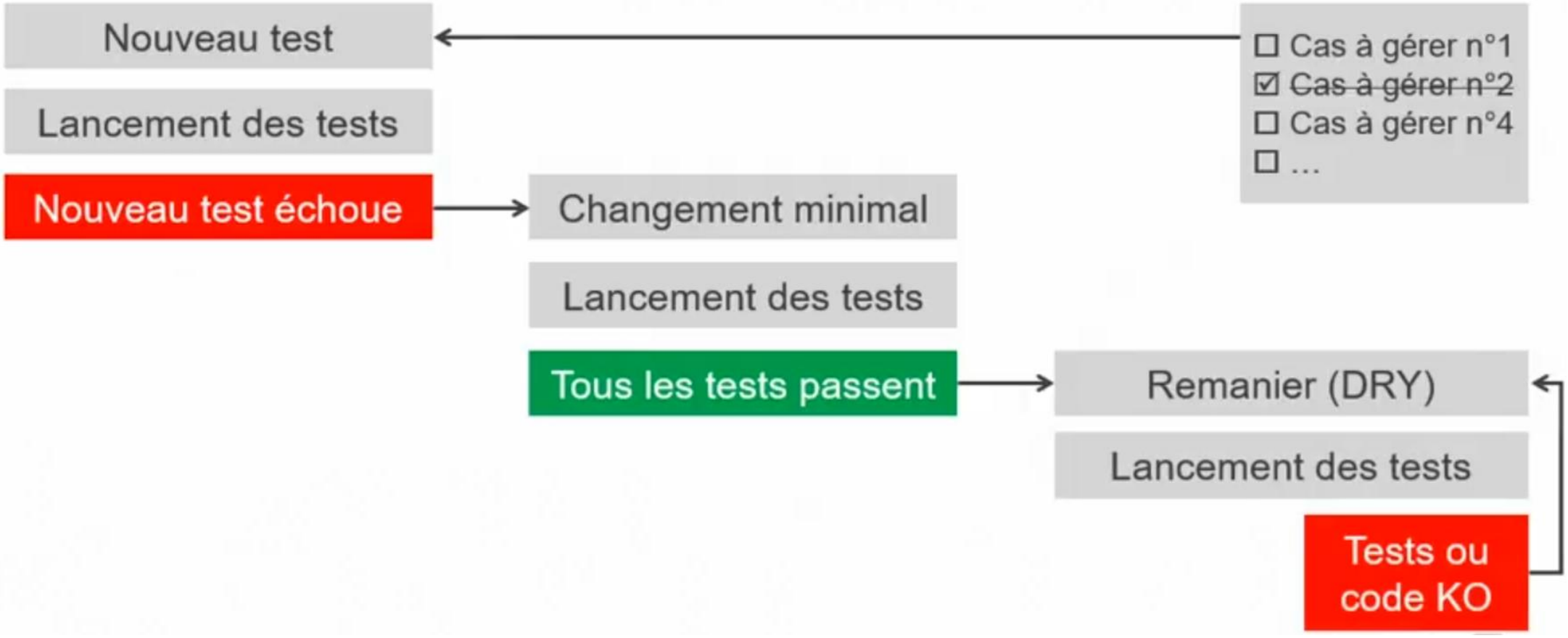
Red


Nouveau test
Lancement des tests
Nouveau test échoue
☐ Cas à gérer n°1
☑ ~~Cas à gérer n°2~~
☐ Cas à gérer n°4
☐ …

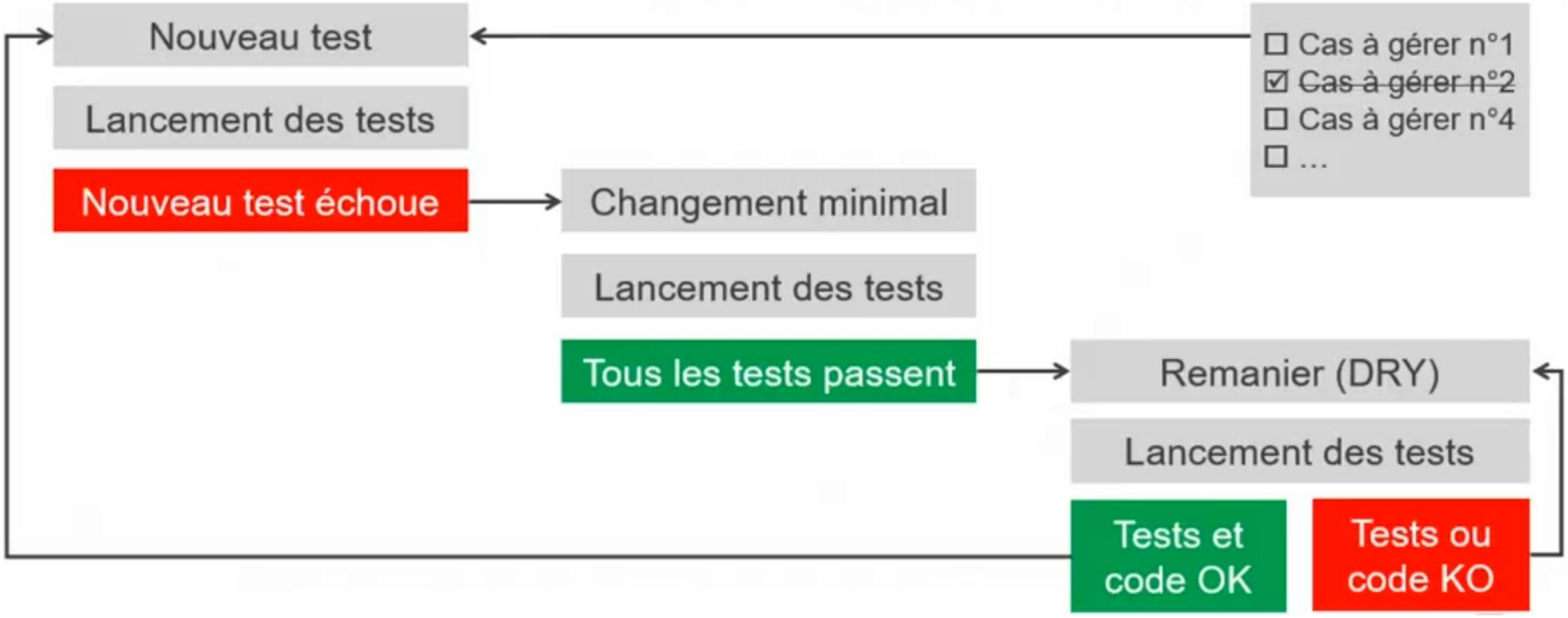
Red Green


Nouveau test
Lancement des tests
Nouveau test échoue
Changement minimal
Lancement des tests
Tous les tests passent
☐ Cas à gérer n°1
☑ Cas à gérer n°2
☐ Cas à gérer n°4
☐ …

Red Green Refactor



Red Green Refactor



TDD

- ✓ Principes
- ✓ Red Green Refactor
- ➔ Emergence
- Synthèse

Emergence

Conception « a priori »

Approche théorique

Nécessité d’anticiper une solution

Découpage « a priori »

Conception « itérative »

Complexification progressive du problème

Emergence d’un découpage objet

TDD

- ✓ Principes
- ✓ Red Green Refactor
- ✓ Emergence
- ➔ Synthèse

Atelier 3 – Heure pour assistant vocal

Sujet

Vous devez développer le module de transcription de l'heure pour un assistant vocal, l'heure doit dont être exprimée de la façon la plus "humaine" possible à l’aide de TDD. Lire le détail du sujet sur 360 Learning.

Livrable

Deux .cs, le test et la classe testée.

RESUME

Avantages de la méthode

Etapes du processus de développement par les tests

Elaboration progressive d’une solution

DEPENDANCES

Objectifs

Classification des dépendances

Traitements à appliquer

Simulation des dépendances

Validation des interactions

DEPENDANCES

➔ Problématique

Prérequis

Stubs vs Mocks

Synthèse

Problematique

Rappels test unitaire

Déterminisme

Isolation de la classe

Dépendances

Appels systèmes : Heure, Aléatoire, Fichiers, …

Instanciation de classes

Champs et paramètres

Appel de méthodes statiques/non

DEPENDANCES

✓ Problématique

➔ Prérequis

Stubs vs Mocks

Synthèse

PREREQUIS

Interfaces, interfaces, interfaces, …

Toute dépendance est remplacée par une interface

En paramètre du constructeur, des méthodes ou injectée

Respect de soLID

Substitution de Liskov : Remplacement par une impl. sans pb

Ségrégation des Interfaces : Plusieurs interfaces / 1 générale

Inversion des Dépendances : dépendre d'abstractions/impl.

DEPENDANCES

- ✓ Problématique
- ✓ Prérequis
- ➔ Stubs vs Mocks
- Synthèse

STUBS / BOUCHON

(Parfois appelé abusivement mock)

Objectifs

Simuler une dépendance en contrôlant son comportement

Concrètement : Définir ce que retourneront les méthodes

Syntaxe

```
var stub = new Mock<IHasard>();
stub.Setup(h => h.Tirer(1, 6)).Returns(2);
/ stub.SetupSequence(h => h.Tirer(1, 6)).Returns(2).Returns(5);
```

Mocks

En plus des stubs

Assertion sur une interaction correcte avec la dépendance

Stub : fait fonctionner la classe testée / Mock : vérifie en +

Syntaxe

```
var mock = new Mock<Serveur>();
mock.Setup(s => s.Envoyer(It.Any<Requete>())).Returns(true);
… // Act
mock.Verify(s => s.Envoyer(…), Times.Once()); // Assert
```

DEPENDANCES

- ✓ Taxonomie
- ✓ Stubs
- ✓ Mocks
- ➔ Synthèse

RESUME

Classification des dépendances

Traitements à appliquer

Simulation des dépendances

BILAN

VOUS SAVEZ MAINTENANT …

Enumérer les critères de bons tests unitaires

Valider le bon fonctionnement d'une classe

Vérifier qu'une classe gère bien les cas d'erreurs

Simuler les dépendances d'une classe

Faire émerger une conception grâce à TDD